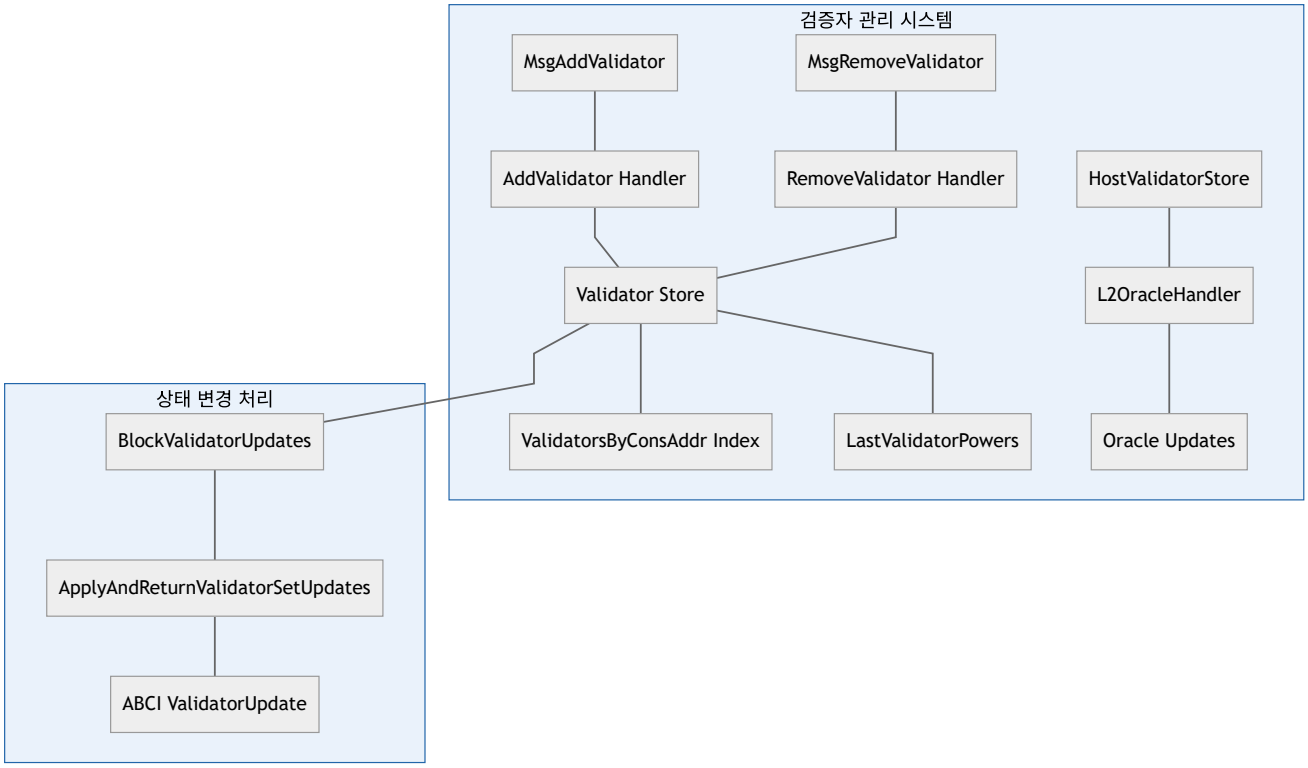
05. 검증자 관리 시스템 분석

개요

OPinit의 opchild 모듈은 L2 체인에서 검증자(Validator) 관리를 담당하는 핵심 시스템을 제공합니다. 이 시스템은 검증자의 추가, 제거, 상태 관리 및 권한 제어를 통해 L2 네트워크의 보안과 합의를 보장합니다.

검증자 관리 아키텍처

핵심 컴포넌트



MsgAddValidator 함수 동작 분석

메시지 구조

```
type MsgAddValidator struct {
Moniker  string
Authority  string
ValidatorAddress  string
```

```
Pubkey *codectypes.Any

}
```

AddValidator 핸들러 구현

`x/opchild/keeper/msg_server.go` 의 `AddValidator` 함수는 다음과 같은 단계로 동작합니다:

```
func (ms MsgServer) AddValidator(ctx context.Context, req *types.MsgAddValidator

// 1. 메시지 검증

if err := req.Validate(ms.authKeeper.AddressCodec(), ms.validatorAddressCodec);

return nil, err

}


// 2. 권한 확인 - authority만 검증자 추가 가능

if ms.authority != req.Authority {

return nil, errorsmod.Wrapf(govtypes.ErrInvalidSigner, "invalid authority; expe

}


// 3. 최대 검증자 수 확인

allValidators, err := ms.GetAllValidators(ctx)

if err != nil {

return nil, err

}
```

```
numMaxValidators, err := ms.MaxValidators(ctx)
if err != nil {
return nil, err
}

if int(numMaxValidators) <= len(allValidators) {
return nil, types.ErrMaxValidatorsExceeded
}

// 4. 중복 검증자 확인
valAddr, err := ms.Keeper.validatorAddressCodec.StringToBytes(req.ValidatorAddre
if err != nil {
return nil, err
}

if _, found := ms.GetValidator(ctx, valAddr); found {
return nil, types.ErrValidatorOwnerExists
}

// 5. 중복 공개키 확인
pk, ok := req.Pubkey.GetCachedValue().(cryptotypes.PubKey)
if !ok {
return nil, errorsmod.Wrapf(sdkerrors.ErrInvalidType, "Expecting cryptotypes.Pu
```

```
}

if _, found := ms.GetValidatorByConsAddr(ctx, sdk.GetConsAddress(pk)); found {
return nil, types.ErrValidatorPubKeyExists
}

// 6. 검증자 생성 및 저장
validator, err := types.NewValidator(valAddr, pk, req.Moniker)
if err != nil {
return nil, err
}

if err := ms.SetValidator(ctx, validator); err != nil {
return nil, err
}
if err = ms.SetValidatorByConsAddr(ctx, validator); err != nil {
return nil, err
}

// 7. 이벤트 발생
sdkCtx.EventManager().EmitEvents(sdk.Events{
sdk.NewEvent(
types.EventTypeAddValidator,
```

```
sdk.NewAttribute(types.AttributeKeyValidator, req.ValidatorAddress),
),
})

return &types.MsgAddValidatorResponse{}, nil
}
```

검증자 생성 과정

1. **권한 검증**: `authority` 주소만이 검증자를 추가할 수 있습니다
2. **용량 확인**: 최대 검증자 수(`MaxValidators`) 제한을 확인합니다
3. **중복 방지**: 동일한 operator address나 consensus public key가 이미 존재하는지 확인합니다
4. **공개키 타입 검증**: 지원되는 공개키 타입인지 확인합니다
5. **저장**: `Validators` 컬렉션과 `ValidatorsByConsAddr` 인덱스에 저장합니다

MsgRemoveValidator 함수 동작 분석

RemoveValidator 핸들러 구현

```
func (ms MsgServer) RemoveValidator(ctx context.Context, req *types.MsgRemoveVal
// 1. 메시지 검증 및 권한 확인
if err := req.Validate(ms.authKeeper.AddressCodec(), ms.validatorAddressCodec);
return nil, err
}
```

```go
if ms.authority != req.Authority {
return nil, errorsmod.Wrapf(govtypes.ErrInvalidSigner, "invalid authority; expe
}

// 2. 검증자 존재 확인
valAddr, err := ms.Keeper.validatorAddressCodec.StringToBytes(req.ValidatorAddre
if err != nil {
return nil, err
}

val, found := ms.Keeper.GetValidator(ctx, valAddr)
if !found {
return nil, errorsmod.Wrap(types.ErrNoValidatorFound, val.OperatorAddress)
}

// 3. 검증자 파워를 0으로 설정 (실제 제거는 val_state_change에서 처리)
val.ConsPower = 0

if err := ms.Keeper.SetValidator(ctx, val); err != nil {
return nil, err
}
```

```go
// 4. 이벤트 발생
sdkCtx.EventManager().EmitEvents(sdk.Events{
sdk.NewEvent(
types.EventTypeRemoveValidator,
sdk.NewAttribute(types.AttributeKeyValidator, req.ValidatorAddress),
),
})

return &types.MsgRemoveValidatorResponse{}, nil
}
```

검증자 제거 메커니즘

검증자 제거는 2단계 프로세스로 진행됩니다:

1. **즉시 단계**: `ConsPower` 를 0으로 설정
2. **지연 단계**: `val_state_change.go` 의 `ApplyAndReturnValidatorSetUpdates` 에서 실제 제거

```go
// val_state_change.go에서의 실제 제거 로직
if validator.ConsPower > 0 {
return nil, errors.New("deleting validator cannot have positive power")
}

if err := k.RemoveValidator(ctx, valAddr); err != nil {
```

```
return nil, err
}
```

opchild 모듈의 검증자 상태 관리

Validator 데이터 구조

```
type Validator struct {
Moniker string
OperatorAddress string
ConsensusPubkey *codectypes.Any
ConsPower int64 // 합의 파워 (항상 1 또는 0)
}
```

상태 저장 컬렉션

opchild 모듈은 다음과 같은 컬렉션을 사용하여 검증자 상태를 관리합니다:

```
type Keeper struct {
// 검증자 정보 저장
Validators collections.Map[[]byte, types.Validator]
ValidatorsByConsAddr collections.Map[[]byte, []byte]
LastValidatorPowers collections.Map[[]byte, int64]
// 호스트 체인 검증자 정보
HostValidatorStore *HostValidatorStore
```

```
// 오라클 핸들러
l2OracleHandler *L2OracleHandler
}
```

검증자 조회 함수

```
// 운영자 주소로 검증자 조회
func (k Keeper) GetValidator(ctx context.Context, addr sdk.ValAddress) (validato
validator, err := k.Validators.Get(ctx, addr)
if errors.Is(err, collections.ErrNotFound) {
return validator, false
} else if err != nil {
panic(err)
}
return validator, true
}

// 합의 주소로 검증자 조회
func (k Keeper) GetValidatorByConsAddr(ctx context.Context, consAddr sdk.ConsAdd
opAddr, err := k.ValidatorsByConsAddr.Get(ctx, consAddr)
if errors.Is(err, collections.ErrNotFound) {
return validator, false
} else if err != nil {
```

```
panic(err)
}
return  k.GetValidator(ctx, opAddr)
}
```

HostValidatorStore와 L2OracleHandler 연동 분석

HostValidatorStore 구조

```
type HostValidatorStore struct {
lastHeight collections.Item[int64]
validators collections.Map[[]byte, stakingtypes.Validator]
consensusAddressCodec address.Codec
}
```

호스트 검증자 업데이트 메커니즘

```
func (k Keeper) UpdateHostValidatorSet(ctx context.Context, clientID  string, he
if  clientID == "" {
return  nil
}

// L1 클라이언트 ID 확인
```

```go
if l1ClientId, err := k.L1ClientId(ctx); err != nil {
return err
} else if l1ClientId != clientID {
return nil
}

return k.HostValidatorStore.UpdateValidators(ctx, height, validatorSet)
}
```

L2OracleHandler와의 연동

```go
type L2OracleHandler struct {
*Keeper
oracleKeeper types.OracleKeeper
extendedCommitCodec connectcodec.ExtendedCommitCodec
veCodec connectcodec.VoteExtensionCodec
}

func (k L2OracleHandler) UpdateOracle(ctx context.Context, height uint64, extCo
// 1. 호스트 검증자 스토어에서 최신 높이 확인
hostStoreLastHeight, err := k.HostValidatorStore.GetLastHeight(ctx)
if err != nil {
```

```
if errors.Is(err, collections.ErrNotFound) {
return types.ErrOracleValidatorsNotRegistered
}
return err
}

// 2. 높이 검증 (최대 5블록 차이 허용)
h := int64(height)
if hostStoreLastHeight-5 > h {
return types.ErrInvalidOracleHeight
}

// 3. 확장 커밋 정보 검증
extendedCommitInfo, err := k.extendedCommitCodec.Decode(extCommitBz)
if err != nil {
return err
}

// 4. 호스트 검증자를 사용한 투표 확장 검증
extendedVotes, err := l2connect.ValidateVoteExtensions(sdkCtx, k.HostValidatorSt
if err != nil {
return err
}
```

```
// 5. 오라클 가격 업데이트
votes, err := l2connect.GetOracleVotes(k.veCodec, extendedVotes)
if err != nil {
return err
}

// ... 가격 집계 및 업데이트 로직
}
```

검증자 권한 및 보안 메커니즘

권한 기반 접근 제어

1. **Authority 권한**: 검증자 추가/제거는 `authority` 주소만 가능
2. **Bridge Executor 권한**: 브리지 관련 작업은 등록된 bridge executor만 가능
3. **Admin 권한**: 파라미터 변경은 admin 주소만 가능

권한 확인 함수

```
// Admin 권한 확인
func (ms MsgServer) checkAdminPermission(ctx context.Context, sender string) er
params, err := ms.GetParams(ctx)
if err != nil {
```

```
return err
}

if params.Admin != sender {
return errorsmod.Wrapf(sdkerrors.ErrUnauthorized, "the message is allowed to be
}
return nil
}

// Bridge Executor 권한 확인
func (ms MsgServer) checkBridgeExecutorPermission(ctx context.Context, sender s
senderAddr, err := ms.authKeeper.AddressCodec().StringToBytes(sender)
if err != nil {
return err
}

bridgeExecutors, err := ms.BridgeExecutors(ctx)
if err != nil {
return err
}
isIncluded := false
for _, bridgeExecutor := range bridgeExecutors {
if bytes.Equal(bridgeExecutor, senderAddr) {
```

```
isIncluded = true
}
}
if !isIncluded {
return  errorsmod.Wrapf(sdkerrors.ErrUnauthorized, "expected included in %s, got
}
return  nil
}
```

보안 검증 메커니즘

1. **중복 방지**: 동일한 operator address나 consensus public key 중복 등록 방지
2. **최대 검증자 수 제한**: `MaxValidators` 파라미터로 검증자 수 제한
3. **공개키 타입 검증**: 지원되는 공개키 타입만 허용
4. **높이 기반 검증**: 오라클 업데이트 시 높이 차이 제한 (최대 5블록)

상태 변경 안전성

```
func (k Keeper) BlockValidatorUpdates(ctx context.Context) ([]abci.ValidatorUpda
// 매 블록 끝에서 검증자 세트 업데이트 계산
updates, err := k.ApplyAndReturnValidatorSetUpdates(ctx)
if  err != nil {
return  nil, err
}
```

```
return  updates, nil
}
```

검증자 상태 변경은 다음과 같은 안전 장치를 가집니다:

1. **원자적 업데이트**: 모든 검증자 변경사항이 한 번에 적용
2. **파워 검증**: 제거되는 검증자의 파워가 0인지 확인
3. **정렬된 처리**: 결정론적 순서로 검증자 처리

에러 처리 및 이벤트

주요 에러 타입

```
var (
ErrNoValidatorFound = errorsmod.Register(ModuleName, 2, "validator does not exis
ErrValidatorOwnerExists = errorsmod.Register(ModuleName, 3, "validator already e
ErrValidatorPubKeyExists = errorsmod.Register(ModuleName, 4, "validator already
ErrValidatorPubKeyTypeNotSupported = errorsmod.Register(ModuleName, 5, "validato
ErrMaxValidatorsExceeded = errorsmod.Register(ModuleName, 15, "max validators ex
)
```

이벤트 타입

```
const (
EventTypeAddValidator = "add_validator"
```

```
EventTypeRemoveValidator = "remove_validator"
)

// 이벤트 속성
const (
AttributeKeyValidator = "validator"
)
```

실제 사용 예시

CLI 명령어 예시

```
# 검증자 추가
opinitd tx opchild add-validator \
--moniker="validator1" \
--validator-address="init1..." \
--pubkey='{"@type":"/cosmos.crypto.ed25519.PubKey","key":"..."}' \
--from=authority

# 검증자 제거
opinitd tx opchild remove-validator \
--validator-address="init1..." \
```

```
--from=authority
```

검증자 조회

```
# 모든 검증자 조회
opinitd query opchild validators

# 특정 검증자 조회
opinitd query opchild validator init1...
```

결론

OPinit의 검증자 관리 시스템은 다음과 같은 특징을 가집니다:

1. **중앙화된 관리**: Authority 권한을 통한 검증자 추가/제거
2. **안전한 상태 전환**: 2단계 제거 프로세스와 원자적 업데이트
3. **호스트 체인 연동**: HostValidatorStore를 통한 L1 검증자 정보 동기화
4. **오라클 통합**: L2OracleHandler를 통한 가격 피드 검증
5. **권한 기반 보안**: 다층 권한 체계로 시스템 보안 보장

이러한 설계를 통해 OPinit은 L2 네트워크의 검증자 세트를 안전하고 효율적으로 관리할 수 있습니다.